Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:01- Implement multi-threaded client/server Process communication using RMI.

AddServer.java

```
import java.rmi.Naming;
public class AddServer {
  public static void main(String args[]) {
    try {
      AddServerImpl addServerImpl = new AddServerImpl();
      Naming.rebind("//localhost/AddServer", addServerImpl);
      System.out.println("Server is running...");
    } catch (Exception e) {
      System.out.println("Exception in main: " + e.getMessage());
      e.printStackTrace();
    }
}
}
```

AddClient.java

```
import  java.rmi.Naming;
import   java.util.Scanner;
public class AddClient {
  public static void main(String args[]) {
    try {
      Scanner sc = new Scanner(System.in);
      // Ask for server IP
      System.out.print("Enter Server IP (e.g., localhost): ");
      String serverIP = sc.nextLine();
      String addServerURL = "//" + serverIP + "/AddServer";
      AddServerIntf  addServerIntf = (AddServerIntf) Naming.lookup(addServerURL);
      // Take numbers as input
      System.out.print("Enter first number: ");
      double d1 = sc.nextDouble();
      System.out.print("Enter second number: ");
      double d2 = sc.nextDouble();
      // Call remote method
      double result = addServerIntf.add(d1, d2);
      System.out.println("The sum is: " + result);
      sc.close();
```

```
    } catch (Exception e) {
      System.out.println("Exception in main: " + e.getMessage());
      e.printStackTrace();
    }
  }
}
```

AddServerIntf.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface AddServerIntf extends Remote{
  double add(double d1, double d2) throws RemoteException;
}
```

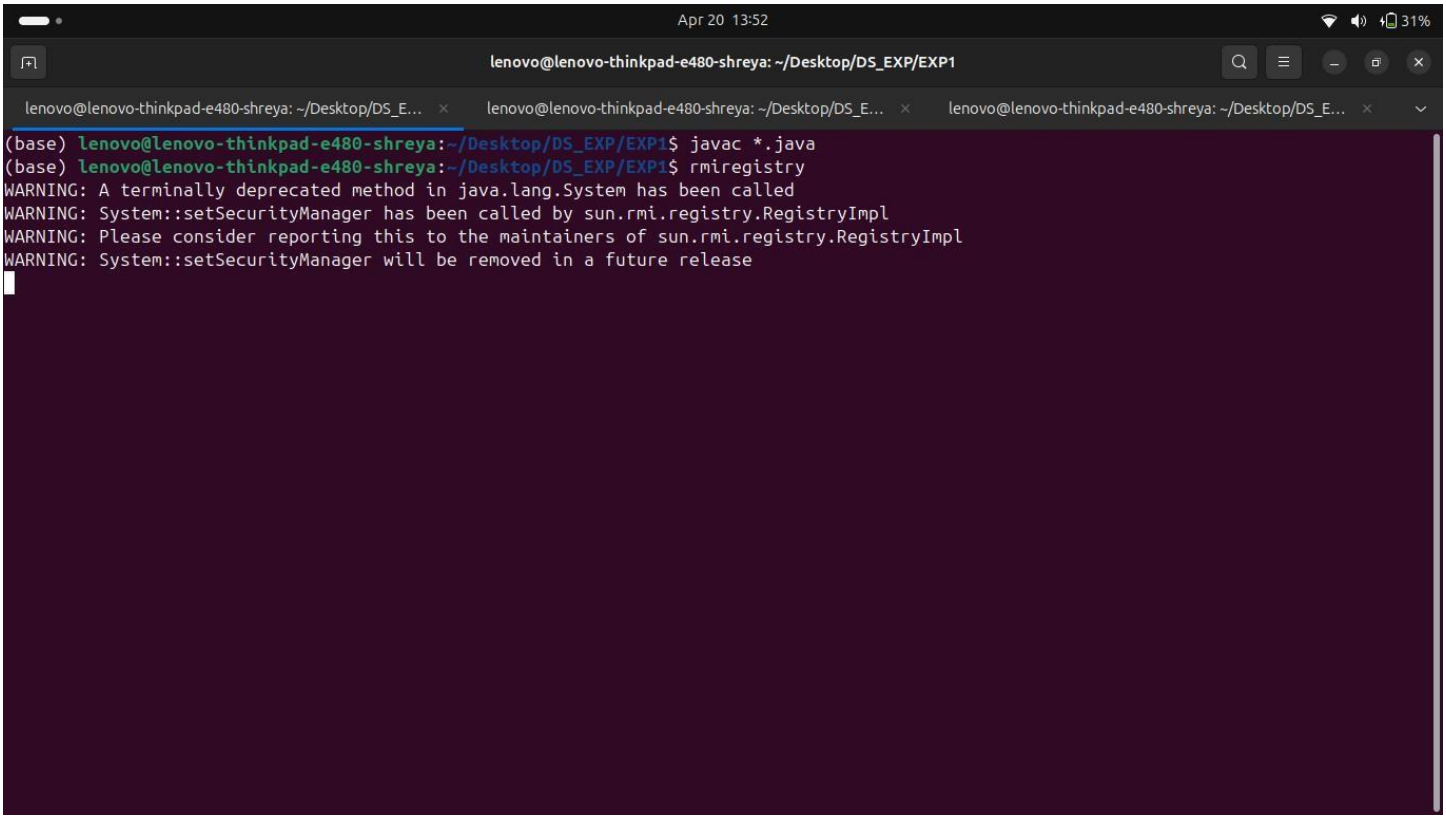
AddServerImpl.java

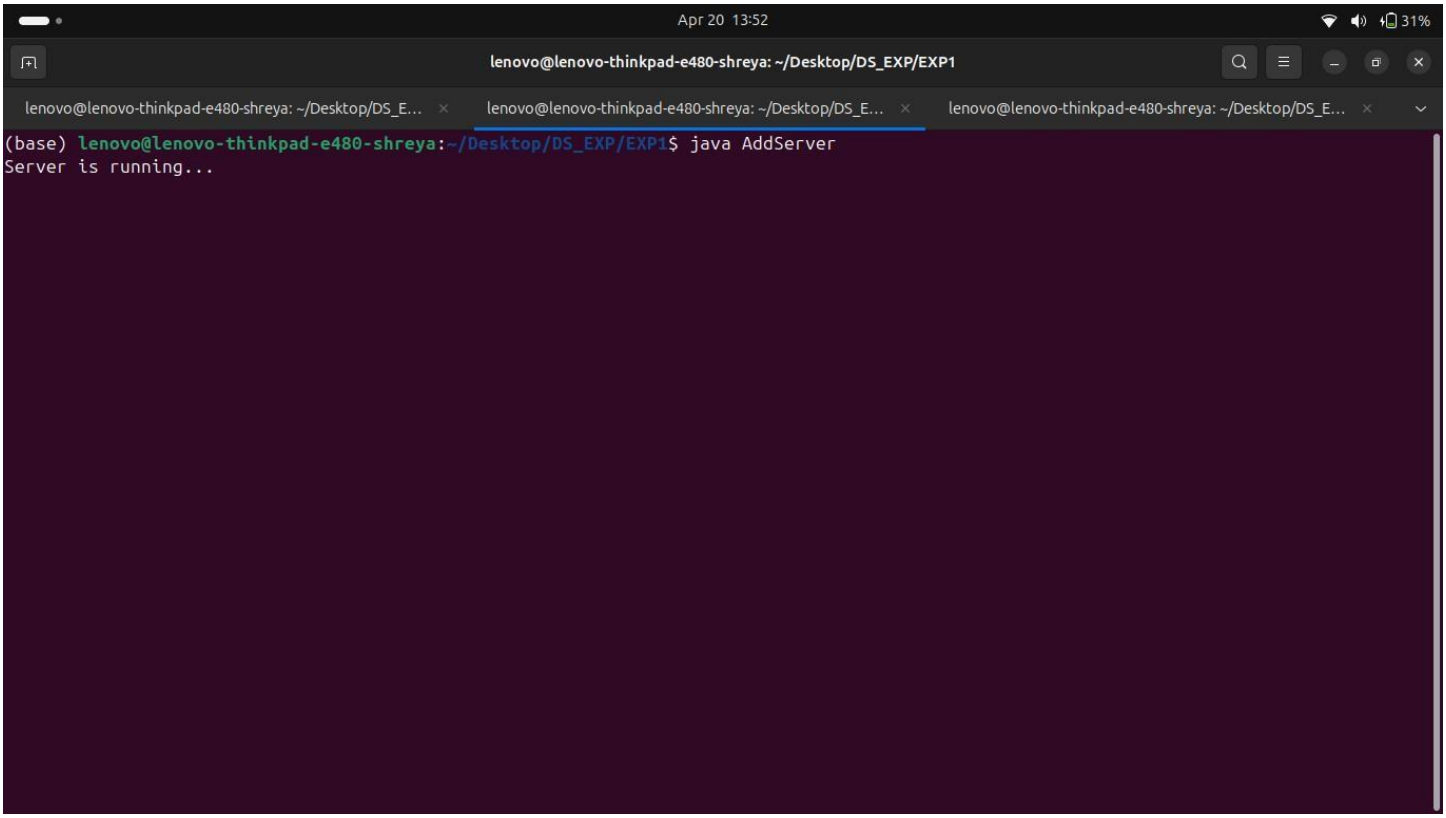
```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
public class AddServerImpl extends UnicastRemoteObject implements AddServerIntf {
  public AddServerImpl() throws RemoteException {
    super();
  }
  public double add(double d1, double d2) throws RemoteException {
    return d1 + d2;
  }
}
```

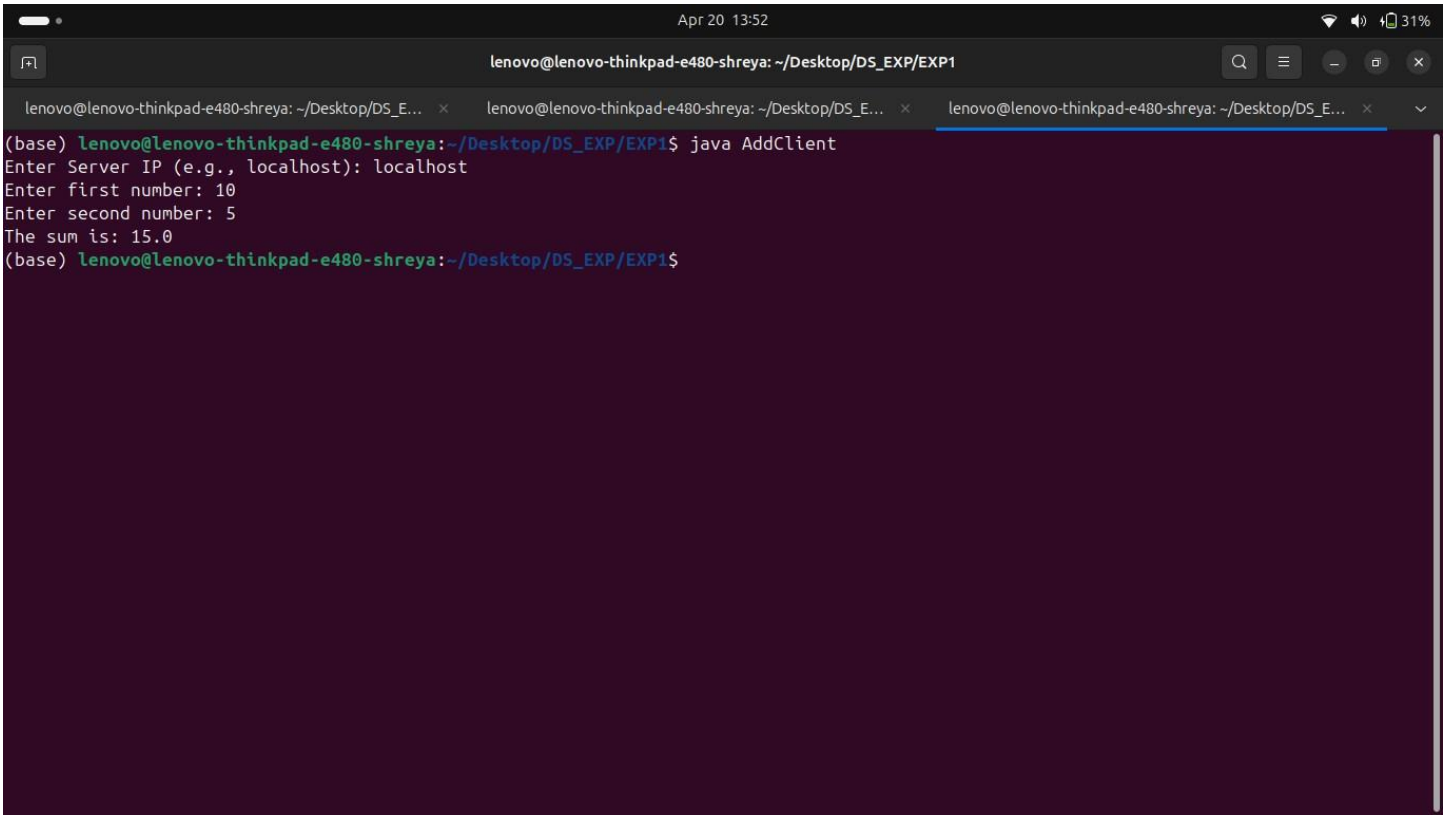
Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:01- Implement multi-threaded client/server Process communication using RMI.

Output:




Apr 20 13:52
31%
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_EXP/EXP1
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
(base) lenovo@lenovo-thinkpad-e480-shreya:~/Desktop/DS_EXP/EXP1$ java AddServer
Server is running...


Apr 20 13:52
31%
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_EXP/EXP1
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
(base) lenovo@lenovo-thinkpad-e480-shreya:~/Desktop/DS_EXP/EXP1$ java AddClient
Enter Server IP (e.g., localhost): localhost
Enter first number: 10
Enter second number: 5
The sum is: 15.0
(base) lenovo@lenovo-thinkpad-e480-shreya:~/Desktop/DS_EXP/EXP1$

Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:02- Develop any distributed application using CORBA to demonstrate object brokering. (Calculator or String operations).

CalculatorApp.idl:

```
module CalculatorApp {
  interface Calculator {
    float add(in float a, in float b);
    float subtract(in float a, in float b);
    float multiply(in float a, in float b);
    float divide(in float a, in float b);
  };
};
```

CalculatorClient.java:

```
import CalculatorApp.*;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;

public class CalculatorClient {
  public static void main(String[] args) {
    try {
      // Create and initialize the ORB
      ORB orb = ORB.init(args, null);

      // Get the root naming context
      org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
      NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);

      // Resolve the object reference in naming
      String name = "Calculator";
      Calculator calculator = CalculatorHelper.narrow(ncRef.resolve_str(name));

      // Perform operations
      float a = 10, b = 5;
      System.out.println("Addition: " + a + " + " + b + " = " + calculator.add(a, b));
      System.out.println("Subtraction: " + a + " - " + b + " = " + calculator.subtract(a, b));
      System.out.println("Multiplication: " + a + " * " + b + " = " + calculator.multiply(a, b));
      System.out.println("Division: " + a + " / " + b + " = " + calculator.divide(a, b));
```

```
    } catch (Exception e) {
      System.err.println("Error: " + e);
      e.printStackTrace(System.out);
    }
  }
}
```

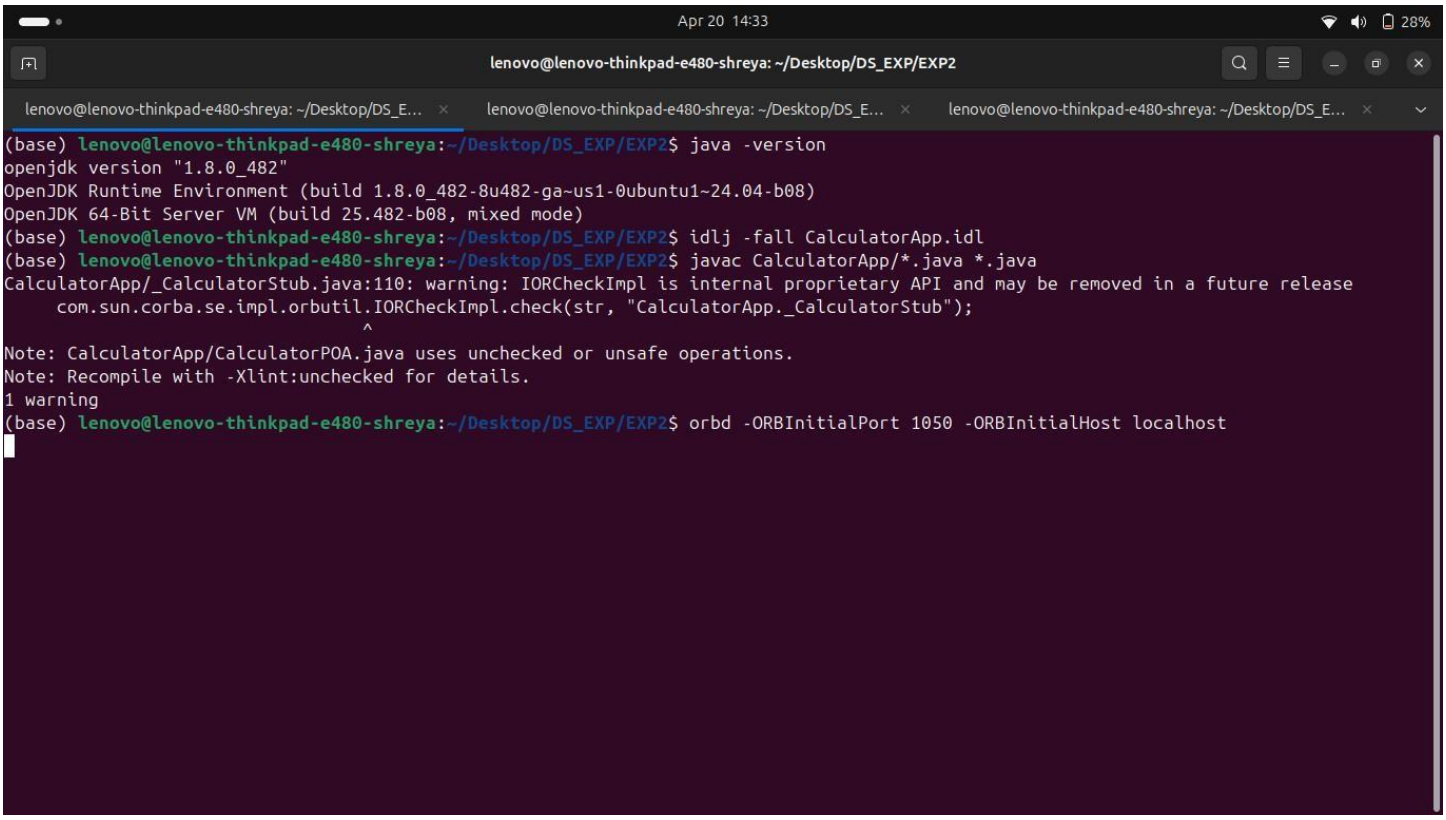
CalculatorServer.java:

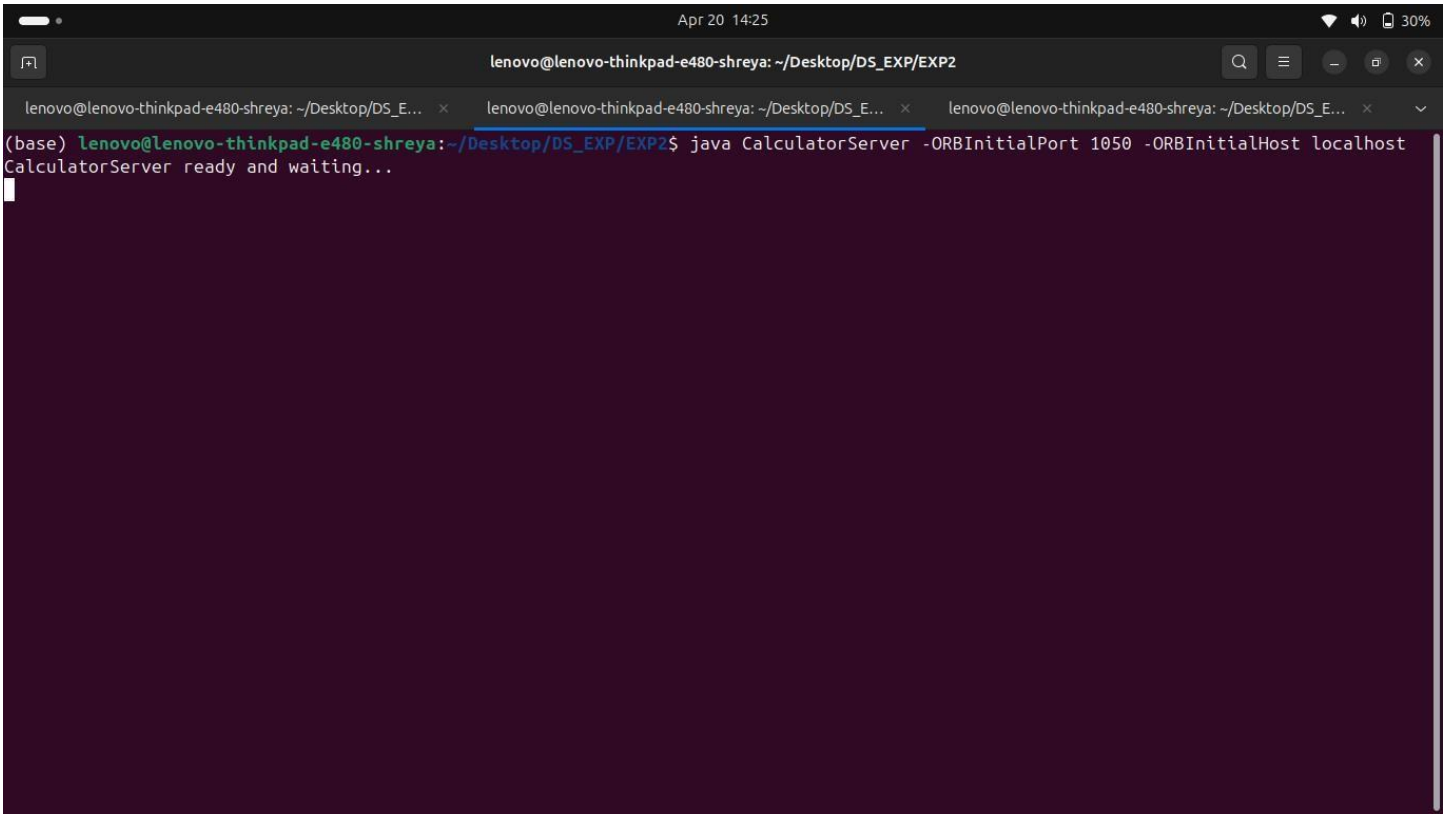
```
import CalculatorApp.*;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;

class CalculatorImpl extends CalculatorPOA {
  public float add(float a, float b) { return a + b; }
  public float subtract(float a, float b) { return a - b; }
  public float multiply(float a, float b) { return a * b; }
  public float divide(float a, float b) {
    if (b == 0) throw new ArithmeticException("Division by zero");
    return a / b;
  }}
public class CalculatorServer {
  public static void main(String args[]) {
    try {
      ORB orb = ORB.init(args, null);
      POA rootpoa = POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
      rootpoa.the_POAManager().activate();
      CalculatorImpl calcImpl = new CalculatorImpl();
      org.omg.CORBA.Object ref = rootpoa.servant_to_reference(calcImpl);
      Calculator href = CalculatorHelper.narrow(ref);
      org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
      NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
      String name = "Calculator";
      NameComponent path[] = ncRef.to_name(name);
      ncRef.rebind(path, href);
      System.out.println("CalculatorServer ready and waiting...");
      orb.run();
    } catch (Exception e) {
      e.printStackTrace();
    }
  }}
```

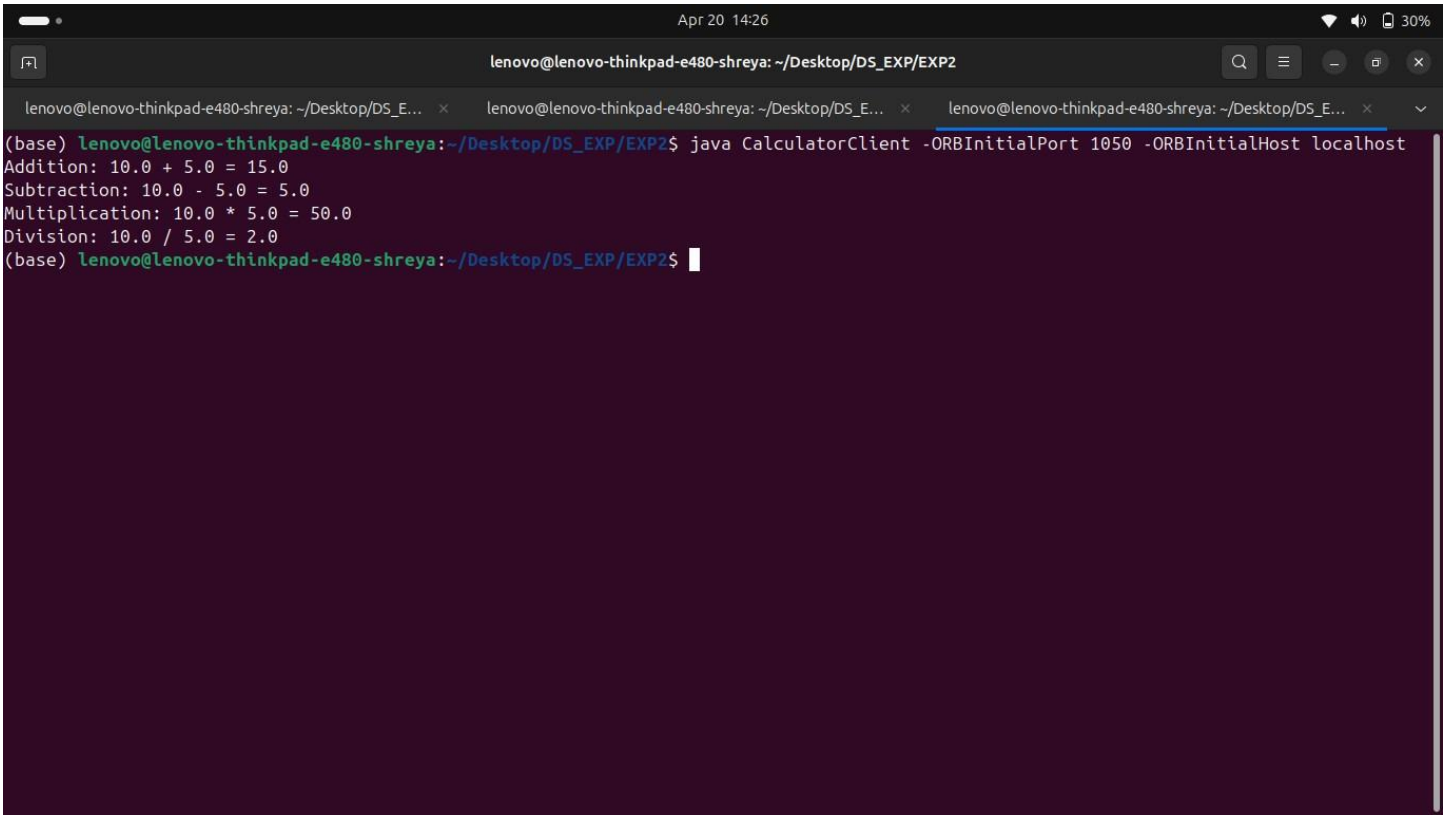
Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:02- Develop any distributed application using CORBA to demonstrate object brokering. (Calculator or String operations).

Output:




Apr 20 14:25
30%
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_EXP/EXP2
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
(base) lenovo@lenovo-thinkpad-e480-shreya:~/Desktop/DS_EXP/EXP2$ java CalculatorServer -ORBInitialPort 1050 -ORBInitialHost localhost
CalculatorServer ready and waiting...


Apr 20 14:26
30%
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_EXP/EXP2
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_E...
(base) lenovo@lenovo-thinkpad-e480-shreya:~/Desktop/DS_EXP/EXP2$ java CalculatorClient -ORBInitialPort 1050 -ORBInitialHost localhost
Addition: 10.0 + 5.0 = 15.0
Subtraction: 10.0 - 5.0 = 5.0
Multiplication: 10.0 * 5.0 = 50.0
Division: 10.0 / 5.0 = 2.0
(base) lenovo@lenovo-thinkpad-e480-shreya:~/Desktop/DS_EXP/EXP2$

Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:03- Develop a distributed system, to find sum of N elements in an array by distributing N/n elements to n number of processors MPI or Open MP. Demonstrate by displaying the intermediate sums calculated at different processors.

---

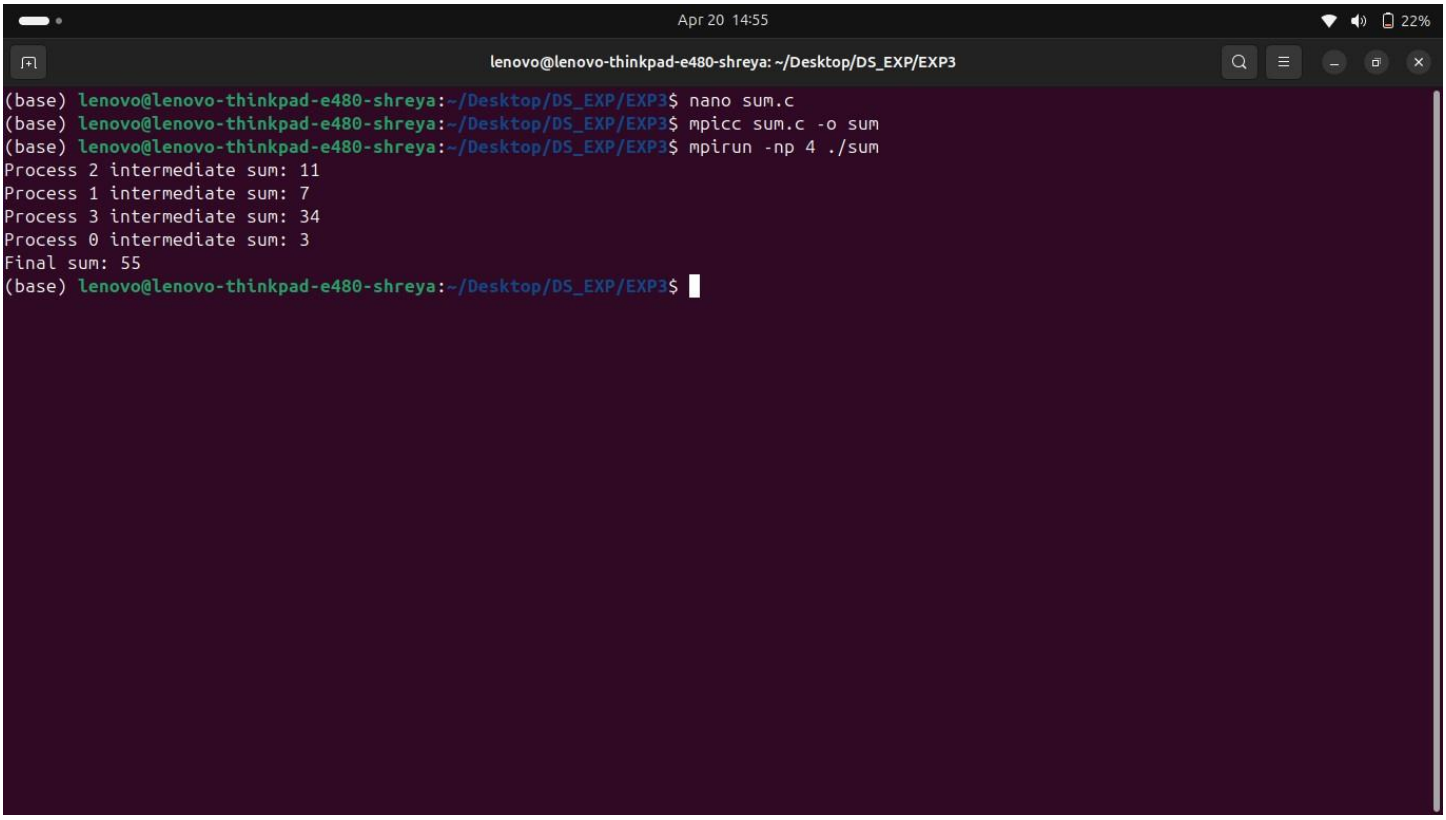
sum.c:

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
  int rank, size;
  int array[10] = {1,2,3,4,5,6,7,8,9,10};
  int localSum = 0, totalSum = 0;
  MPI_Init(Cargc, Cargv);
  MPI_Comm_rank(MPI_COMM_WORLD, Crank);
  MPI_Comm_size(MPI_COMM_WORLD, Csize);
  int chunk = 10 / size;
  int start = rank * chunk;
  int end = (rank == size - 1) ? 10 : start + chunk;
  for(int i = start; i < end; i++) {
    localSum += array[i];
  }
  printf("Process %d intermediate sum: %d\n", rank, localSum);
  MPI_Reduce(ClocalSum, CtotalSum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
  if(rank == 0) {
    printf("Final sum: %d\n", totalSum);
  }
  MPI_Finalize();
  return 0;
}
```

Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:03- Develop a distributed system, to find sum of N elements in an array by distributing N/n elements to n number of processors MPI or Open MP. Demonstrate by displaying the intermediate sums calculated at different processors.

Output:



Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:04- Implement Berkeley algorithm for clock synchronization.

Berkeley.java:

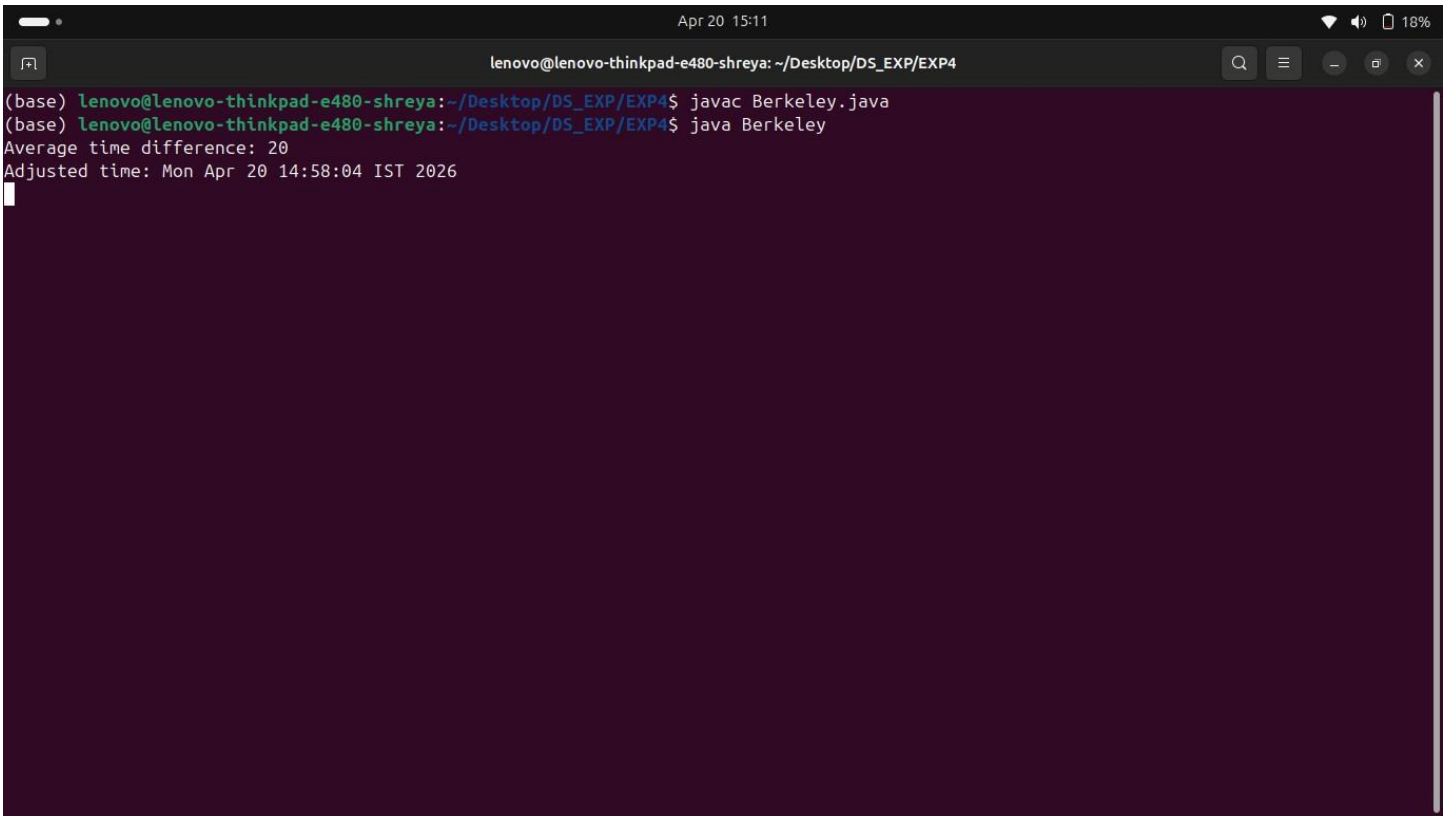
```
import java.io.*;
import java.net.*;
import java.util.*;
public class Berkeley {
  private static final int PORT = 9876;
  public static void main(String[] args) throws Exception {
    ServerSocket serverSocket = new ServerSocket(PORT);
    List<Long> timeDiffs = Collections.synchronizedList(new ArrayList<Long>());
    Thread timeServerThread = new Thread(() -> {
      while (true) {
        try (Socket clientSocket = serverSocket.accept();
          ObjectInputStream in = new ObjectInputStream(clientSocket.getInputStream());
          ObjectOutputStream out = new ObjectOutputStream(clientSocket.getOutputStream())) {
          Date clientTime = (Date) in.readObject();
          out.writeObject(new Date());
          long timeDiff = new Date().getTime() - clientTime.getTime();
          timeDiffs.add(timeDiff);
        } catch (Exception e) {
          e.printStackTrace();
        }
      }
    });
    timeServerThread.start();
    Thread timeClientThread = new Thread(() -> {
      while (true) {
        try (Socket socket = new Socket("localhost", PORT);
          ObjectOutputStream out = new ObjectOutputStream(socket.getOutputStream());
          ObjectInputStream in = new ObjectInputStream(socket.getInputStream())) {
          out.writeObject(new Date());
          Date serverTime = (Date) in.readObject();
          long timeDiff = serverTime.getTime() - new Date().getTime();
          timeDiffs.add(timeDiff);
          Thread.sleep(1000);
        } catch (Exception e) {
          e.printStackTrace();
```

```
            }
        }
    });
    timeClientThread.start();
    Thread.sleep(20000);
    long sumTimeDiff = 0;
    for (Long timeDiff : timeDiffs) {
        sumTimeDiff += timeDiff;
    }
    long avgTimeDiff = sumTimeDiff / timeDiffs.size();
    System.out.println("Average time difference: " + avgTimeDiff);
    adjustClock(avgTimeDiff);
  }
  private static void adjustClock(long avgTimeDiff) {
    Calendar calendar = Calendar.getInstance();
    calendar.setTime(new Date());
    calendar.add(Calendar.MILLISECOND, (int) avgTimeDiff);
    System.out.println("Adjusted time: " + calendar.getTime());
  }
}
```

Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:04- Implement Berkeley algorithm for clock synchronization.

Output:



Name: Shreya Balakrishna Ghodake
Roll no: 407A031
Subject: Distributed Systems

Experiment no:05- Implement token ring based mutual exclusion algorithm.

TokenRing.java:

```
import java.util.*;
public class TokenRing {
  static final int NUM_NODES = 3;
  static int token = 0;
  static class Process extends Thread {
    int id;
    public Process(int id) {
      this.id = id;
    }
    public void run() {
      try {
        while (true) {
          Thread.sleep((int)(Math.random() * 3000));
           if (token == id) {
            // Enter critical section
            System.out.println("Process " + id + " ENTERING CS");
             Thread.sleep(1000);
            System.out.println("Process " + id + " EXITING CS");
             // Pass token
            token = (id + 1) % NUM_NODES;
            System.out.println("Token passed to Process " + token);
          }
        }
      } catch (Exception e) {
        e.printStackTrace();
      }
    }
  }
  public static void main(String[] args) {
    for (int i = 0; i < NUM_NODES; i++) {
      new Process(i).start();
    }
  }
}
```

Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:05- Implement token ring based mutual exclusion algorithm.

Output:

```
lenovo@lenovo-thinkpad-e480-shreya: ~/Desktop/DS_EXP/EXP5
(base) lenovo@lenovo-thinkpad-e480-shreya:~/Desktop/DS_EXP/EXP5$ javac TokenRing.java
(base) lenovo@lenovo-thinkpad-e480-shreya:~/Desktop/DS_EXP/EXP5$ java TokenRing
Process 0 ENTERING CS
Process 0 EXITING CS
Token passed to Process 1
Process 1 ENTERING CS
Process 1 EXITING CS
Token passed to Process 2
Process 2 ENTERING CS
Process 2 EXITING CS
Token passed to Process 0
Process 0 ENTERING CS
Process 0 EXITING CS
Token passed to Process 1
Process 1 ENTERING CS
Process 1 EXITING CS
Token passed to Process 2
Process 2 ENTERING CS
Process 2 EXITING CS
Token passed to Process 0
Process 0 ENTERING CS
Process 0 EXITING CS
Token passed to Process 1
Process 1 ENTERING CS
Process 1 EXITING CS
Token passed to Process 2
Process 2 ENTERING CS
Process 2 EXITING CS
Token passed to Process 0
Process 0 ENTERING CS
Process 0 EXITING CS
```

Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:06- Implement Bully and Ring algorithm for leader election.

Bully.java:

```
import java.util.*;
public class Bully {
  static int n = 5;
   static boolean[] active = {true, true, true, true, true};
   static int coordinator = 4; // index (0-based), so process 5
   public static void election(int initiator) {
     System.out.println("\nProcess " + (initiator + 1) + " starts election");
     boolean higherAlive = false;
     for (int i = initiator + 1; i < n; i++) {
       if (active[i]) {
         System.out.println("Process " + (initiator + 1) + " sends ELECTION to " + (i + 1));
         higherAlive = true;
       }
     }
     if (!higherAlive) {
       coordinator = initiator;
       System.out.println("Process " + (initiator + 1) + " becomes COORDINATOR");
     } else {
       for (int i = initiator + 1; i < n; i++) {
         if (active[i]) {
           System.out.println("Process " + (i + 1) + " sends OK to " + (initiator + 1));
           election(i); // higher process takes over
           return;
         }
       }
     }
   }
   public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);

     System.out.println("Initial Coordinator: Process " + (coordinator + 1));
     while (true) {
       System.out.println("\n1. Down Process\n2. Start Election\n3. Exit");
       int choice = sc.nextInt();
```

```
        switch (choice) {
          case 1:
            System.out.print("Enter process to down: ");
            int down = sc.nextInt() - 1;
            active[down] = false;
            System.out.println("Process " + (down + 1) + " is DOWN");
            if (down == coordinator) {
              System.out.println("Coordinator failed!");
            }
            break;
           case 2:
            System.out.print("Enter initiator process: ");
            int init = sc.nextInt() - 1;
            election(init);
            System.out.println("New Coordinator: Process " + (coordinator + 1));
            break;
          case 3:
            return;
        }
      }
    }
}
```

RingAlgorithm.java:

```
import java.util.Scanner;
public class RingAlgorithm {
  public static void main(String[] args) {
     int i, j;
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter the number of processes: ");
    int numberOfProcesses = scanner.nextInt();
     Process[] processes = new Process[numberOfProcesses];
    // Object initialization
    for (i = 0; i < processes.length; i++)
      processes[i] = new Process();
    // Getting input from users
    for (i = 0; i < numberOfProcesses; i++) {
      processes[i].index = i;
      System.out.println("Enter the ID of process " + (i + 1) + ": ");
      processes[i].id = scanner.nextInt();
      processes[i].state = "active";
      processes[i].hasSentMessage = false;
```

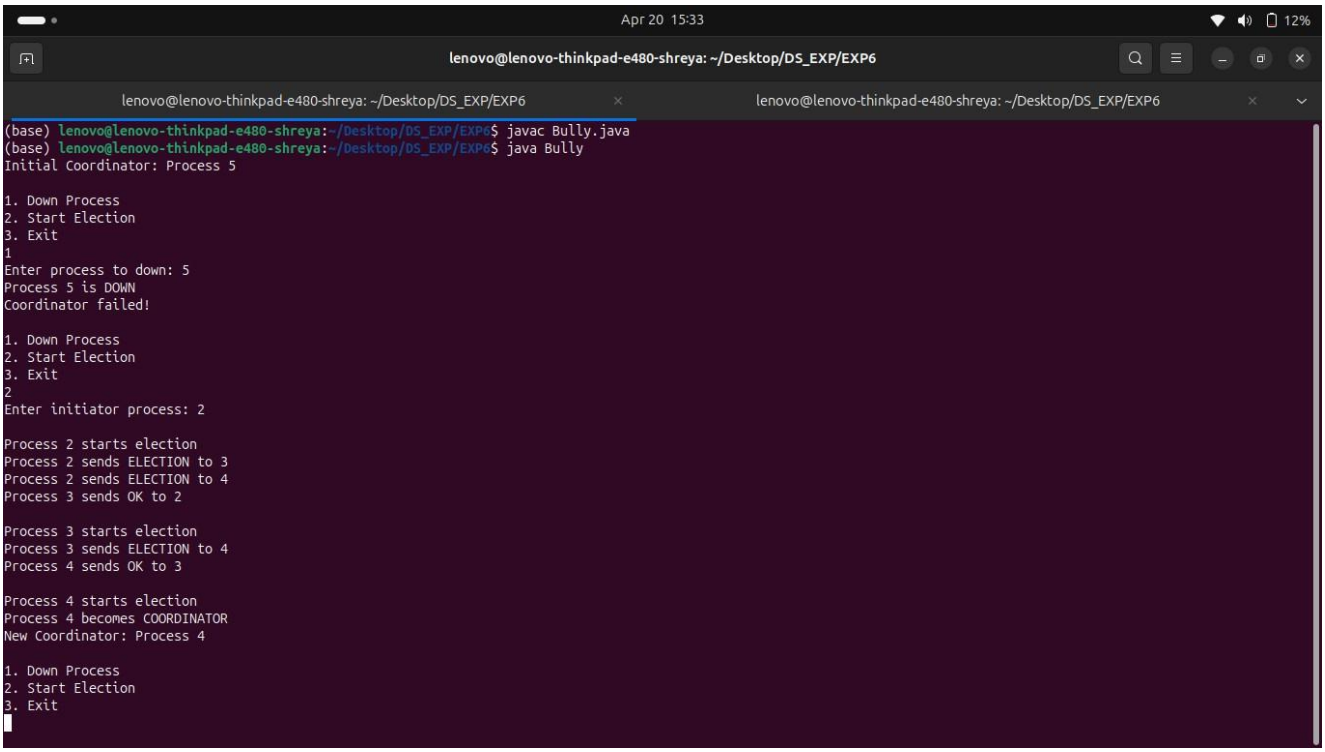
```
    }
    // Sorting the processes based on their IDs
    for (i = 0; i < numberOfProcesses - 1; i++) {
      for (j = 0; j < numberOfProcesses - i - 1; j++) {
        if (processes[j].id > processes[j + 1].id) {
          Process temp = processes[j];
          processes[j] = processes[j + 1];
          processes[j + 1] = temp;
        }
      }
    }
    System.out.println("Processes in sorted order:");
    for (i = 0; i < numberOfProcesses; i++) {
      System.out.println("[" + processes[i].index + "] " + processes[i].id);
    }
    // Initiating coordinator selection
    int initiatorIndex;
    while (true) {
      System.out.println("\n1. Initiate Election\n2. Quit");
      int choice = scanner.nextInt();
      switch (choice) {
        case 1:
          System.out.println("Enter the index of the process initiating the election: ");
          initiatorIndex = scanner.nextInt();
          initiateElection(processes, initiatorIndex, numberOfProcesses);
          break;
        case 2:
          System.out.println("Program terminated.");
          return;
        default:
          System.out.println("Invalid response.");
      }
    }
}
 public static void initiateElection(Process[] processes, int initiatorIndex, int numberOfProcesses) {
    System.out.println("Election initiated by Process " + processes[initiatorIndex].id);
    processes[initiatorIndex].hasSentMessage = true;

    int tempIndex = initiatorIndex;
    int nextIndex = (initiatorIndex + 1) % numberOfProcesses;
    while (nextIndex != initiatorIndex) {
      if (processes[nextIndex].state.equals("active") CC !processes[nextIndex].hasSentMessage) {
```

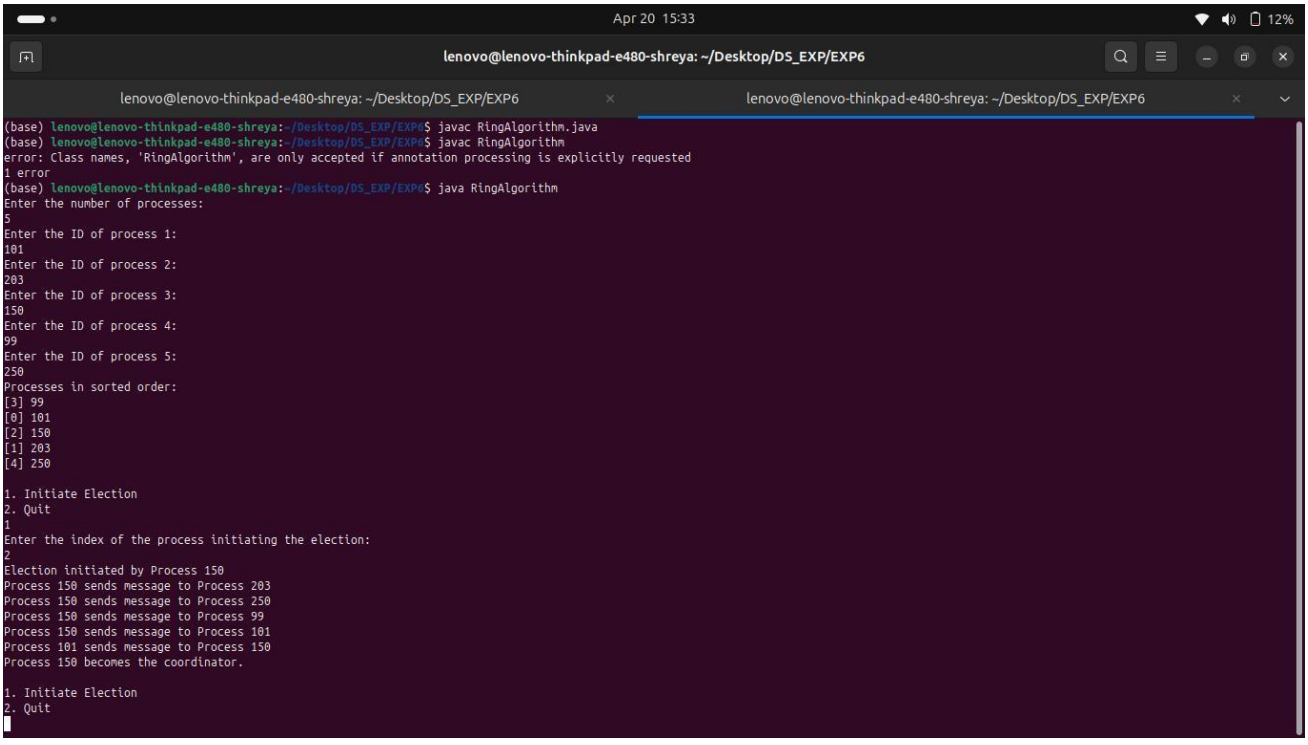
```
            System.out.println("Process " + processes[initiatorIndex].id + " sends message to Process " +
processes[nextIndex].id);
            processes[nextIndex].hasSentMessage = true;
            tempIndex = nextIndex;
          }
          nextIndex = (nextIndex + 1) % numberOfProcesses;
        }
        System.out.println("Process " + processes[tempIndex].id + " sends message to Process " +
processes[initiatorIndex].id);
        System.out.println("Process " + processes[initiatorIndex].id + " becomes the coordinator.");
        processes[initiatorIndex].state = "inactive";
      }
}
class Process {
  public int index;       // Index of the process
  public int id;          // ID of the process
  public boolean hasSentMessage; // Flag to indicate whether the process has sent a message
  public String state;       // Indicates whether the process is active or inactive
}
```

Name: Sanika Deepak Patil
Roll no: 407B026
Subject: Distributed Systems

Experiment no:06- Implement Bully and Ring algorithm for leader election.

Output:





Assignment 07: Create a simple web service and write any distributed - application to consume the

Source Code

CalculatorServer.java

```
import java.io.*;
import java.net.*;

public class CalculatorServer {
    public static void main(String[] args) throws IOException {
        int port = 8080;
        ServerSocket serverSocket = new ServerSocket(port);
        System.out.println("Calculator Web Service at http://localhost:"+port);

        while (true) {
            Socket client = serverSocket.accept();
            new Thread(() -> handleRequest(client)).start();
        }
    }

    static void handleRequest(Socket client) {
        try {
            BufferedReader in = new BufferedReader(
                new InputStreamReader(client.getInputStream()));
            OutputStream out = client.getOutputStream();
            String requestLine = in.readLine();
            if (requestLine == null) { client.close(); return; }

            String[] parts = requestLine.split(" ");
            String path = parts.length > 1 ? parts[1] : "/";

            String response = processCalculation(path);

            String http = "HTTP/1.1 200 OK\r\n"
                + "Content-Type: application/json\r\n"
                + "Content-Length: "+response.length()+"\r\n"
                + "Connection: close\r\n\r\n" + response;
            out.write(http.getBytes());
            out.flush();
            client.close();
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }

    static String processCalculation(String path) {
        try {
            String op = path.split("\\?")[0].substring(1);
            String query = path.contains("?") ? path.split("\\?")[1] : "";
            int a = 0, b = 0;
            for (String p : query.split("&")) {
                String[] kv = p.split("=");
                if (kv[0].equals("a")) a = Integer.parseInt(kv[1]);
                if (kv[0].equals("b")) b = Integer.parseInt(kv[1]);
            }
            double result;
            switch (op) {
                case "add":     result = a + b; break;
                case "subtract": result = a - b; break;
                case "multiply": result = a * b; break;
                case "divide":   result = (double)a / b; break;
```

```
                default: return "{\"error\": \"Unknown\"}";
            }
            return "{\"operation\": \""+op+"\", \"a\": "+a
                +", \"b\": "+b+", \"result\": "+result+"}";
        } catch (Exception e) {
            return "{\"error\": \""+e.getMessage()+"\"}";
        }
    }
}
```

CalculatorClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class CalculatorClient {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;
        do {
            System.out.println("\n=== Calculator Web Service Client ===");
            System.out.println("1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            if (choice >= 1 && choice <= 4) {
                System.out.print("Enter first number: ");
                int a = sc.nextInt();
                System.out.print("Enter second number: ");
                int b = sc.nextInt();

                String op;
                switch (choice) {
                    case 1: op="add"; break;
                    case 2: op="subtract"; break;
                    case 3: op="multiply"; break;
                    default: op="divide";
                }
                System.out.println("Response: "+callService(op, a, b));
            }
        } while (choice != 5);
        sc.close();
    }

    static String callService(String op, int a, int b) {
        try {
            URL url = new URL("http://localhost:8080/"
                +op+"?a="+a+"&b="+b);
            HttpURLConnection c = (HttpURLConnection)url.openConnection();
            c.setRequestMethod("GET");
            BufferedReader in = new BufferedReader(
                new InputStreamReader(c.getInputStream()));
            StringBuilder r = new StringBuilder();
            String line;
            while ((line = in.readLine()) != null) r.append(line);
            in.close(); c.disconnect();
            return r.toString();
        } catch (Exception e) {
            return "Error: " + e.getMessage();
        }
    }
}
```

Input

```
Server Terminal:
$ javac CalculatorServer.java CalculatorClient.java
$ java CalculatorServer

Client Terminal:
$ java CalculatorClient

User Input:
  Choice 1 (Add):       a = 5,  b = 3
  Choice 2 (Subtract):  a = 10, b = 4
  Choice 3 (Multiply):  a = 20, b = 5
  Choice 4 (Divide):    a = 15, b = 3
  Choice 5 (Exit)
```

Output

```
--- Server Terminal ---
Calculator Web Service running at http://localhost:8080
Endpoints: /add?a=X&b=Y  /subtract?a=X&b=Y  /multiply?a=X&b=Y  /divide?a=X&b=Y
Request: GET /add?a=5&b=3 HTTP/1.1
add(5, 3) = 8.0
Request: GET /subtract?a=10&b=4 HTTP/1.1
subtract(10, 4) = 6.0
Request: GET /multiply?a=20&b=5 HTTP/1.1
multiply(20, 5) = 100.0
Request: GET /divide?a=15&b=3 HTTP/1.1
divide(15, 3) = 5.0

--- Client Terminal ---
=== Calculator Web Service Client ===
Enter choice: 1
Enter first number: 5
Enter second number: 3
Response: { "operation": "add", "a": 5, "b": 3, "result": 8.0 }

Enter choice: 2
Enter first number: 10
Enter second number: 4
Response: { "operation": "subtract", "a": 10, "b": 4, "result": 6.0 }

Enter choice: 3
Enter first number: 20
Enter second number: 5
Response: { "operation": "multiply", "a": 20, "b": 5, "result": 100.0 }

Enter choice: 4
Enter first number: 15
Enter second number: 3
Response: { "operation": "divide", "a": 15, "b": 3, "result": 5.0 }

Enter choice: 5
Client exited.
```